

Voluntary Exercise 3*Mon, Apr 16**due Mon, Apr 30*

Problem 1. The *edge connectivity* of an undirected graph is the minimum number k of edges that must be removed to disconnect the graph. For example, the edge connectivity of a tree is 1, and the edge connectivity of a cyclic chain of vertices is 2.

Show how the edge connectivity of an undirected graph $G = (V, E)$ can be determined by running a maximum-flow algorithm on at most $|V|$ flow networks, each having $O(V)$ vertices and $O(E)$ edges. ♣

Solution. Construct a directed graph G^* from G by replacing each edge u, v in G by two directed edge (u, v) and (v, u) in G^* . Let $f^*(u, v)$ be the maximum flow value from u to v through G^* with all edge capacities equal to 1. Pick an arbitrary node u and compute $f^*(u, v)$ for all $v \neq u$.

We claim that edge connectivity equals $c^* = \min_{v \neq u} f^*(u, v)$. Therefore the edge connectivity of G can be computed by running the maximum-flow algorithm $|V| - 1$ times on flow networks each having $|V|$ vertices and $2|E|$ edges.

Suppose k is the edge connectivity of the graph and S is the set of k edges such that removal of S will disconnect the graph into 2 non-empty subgraphs G_1 and G_2 . WLOG assume the node $u \in G_1$. Let w be a node in G_2 . Since $u \neq w$ the value $f^*(u, w)$ will be computed by the algorithm. By the max-flow min-cut theorem, $f^*(u, w)$ equals the min cut size between the pair (u, w) , which is at most k since S disconnects u and w . Therefore, we have

$$c^* \leq f^*(u, w) \leq k$$

But c^* cannot be smaller than k since that would imply a cut set of size smaller than k , contradicting the fact that k is the edge connectivity. Therefore $c^* = k$ and the algorithm returns the edge connectivity of the graph correctly. ■

Problem 3

(a) Seeing page 29 (4/2), because the proof has no concern with this set, so “0” can be arbitrary value.

(b) Claim: 任一個大的 flow 減去任一個小的 flow 後都還會是一個 flow
(規定每一條水管都是大減小)

對於任何一個 flow F , 從起點找一條到終點的 path, 假設這條 path 上每條水管中的水流的最小值是 m , 定義 Flow P 等於這條 path, 流量等於 m , 接著把 F 減去 P , 根據上述 claim 可以得到一個新的 flow N , 則 N 上至少有一條水管的流量會是 0! 重複上述動作, 當 flow F 被減成 0 的時候就停止, 令每次找到的 flow 為 $P_1, P_2, P_3 \dots$, 則 F 就可以被表示成 $P_1 + P_2 + P_3 + \dots$, 因為每次都會有一條水管的流量變成 0, 所以這個動作最多只會有 $|E|$ 次.

既然對於任何一個 flow F 都 work, 所以 maximum flow 當然可以 work!

Problem 3. Give an efficient algorithm to find the length (number of edges) of a minimum length negative-weight cycle in a graph.

Solution.

Consider the naive algorithm SLOW-ALL-PAIRS-SHORTEST-PATHS in chapter 25 of the textbook. The matrix $L^{(k)}$ used in this algorithm is the same as the matrix w_k mentioned in class. Notice that the presence of negative-weight cycle can be determined by looking for a negative value in the diagonal of matrix L . If $L^{(s)}$ is the first time a negative value appears in the diagonal of the matrix, then s is the length of a minimum length negative-weight cycle in the graph.

Problem 4. Assume the number of players is m .

• **Algorithm:**

1. while ($findpath = 1$) {
2. $findpath = 0$;
3. for ($i = 1$ to m) {
4. if ($player_i$ is unlabeled) {
5. Use modified BFS to find a path from $player_i$ to
6. an unlabeled job. Only job label with j can walk
7. to $player_j$.
8. If such a path is found {
9. $findpath = 1$;
10. label the job on this path with j
11. if $player_j$ choose it;
12. }
13. }
14. }
15. }
16. Scan the jobs with label, then we can find all matched pairs.

• **Time:**

1. the while loop runs at most m times.
2. the for loop runs at most m times.
3. the modified BFS need $O(|V| + |E|)$ time.
4. labeling jobs on path need $O(|V| + |E|)$ time.
5. scan jobs need $O(|V|)$ time
6. total running time = $O(|V| * |V| * (2(|V| + |E|)) + |V|) = O(|V|^2|E|)$.

• **Correctness:**

G has a maximum matching if and only if we can't find any alternative path.

onlyif direction :

If we can find an alternative path, there exists a larger matching trivially.

if direction :

By contradiction we assume G has a larger matching M , then there must be some unlabeled players who can find a job after the algorithm. Starting from any of these players, run the modified BFS depends on M , which means when arriving $player_j$, we should walk to the job which is paired with $player_j$ in M . Until we walk to an unlabeled job, we find the alternative path.